NoSQL_Databases Report

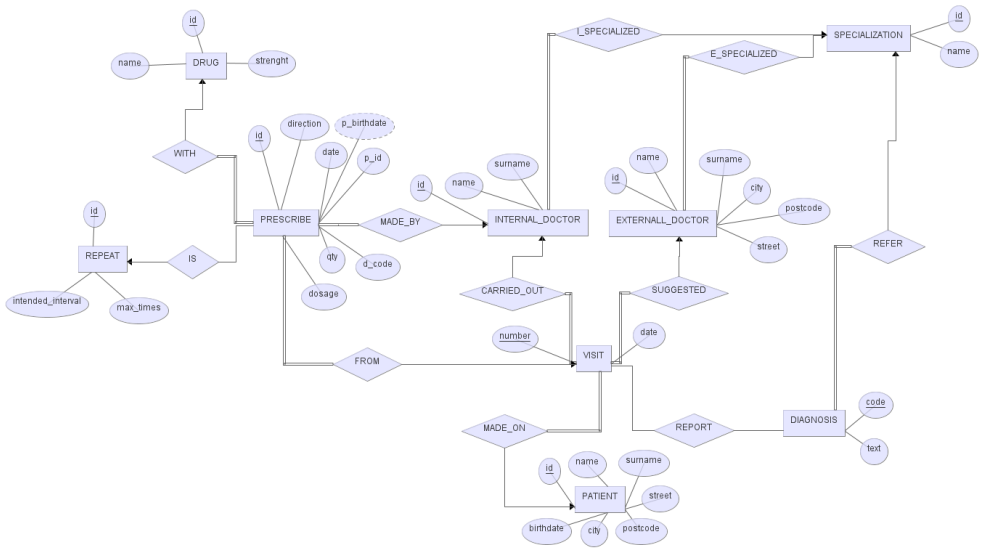
Data Management- 2024-2025

Project Topic: Modify the existing SQL-Medical-Office database and migrate the new data using a NoSQL approach like MongoDB. Finally, analyze and compare the differences between the two approaches.

Student Name: [Sergio Fernandez Diz]

Assignment .1 - Relational schema definition

a) ER-Diagram: The first step is to incorporate the new requirements into the existing ER schema. As a first action, the Entity Sets and their associated Attributes have been identified.Once the entity sets were identified, the schema was created as follows:



We have identified a total of three entity sets. The first is **Prescribe**, which contains all the necessary attributes related to the prescription. Additionally, the **patient's birthdate** (p_birthdate) can be derived from the patient's ID. The second entity is **Drug**, which we decided to represent as a separate set due to the varying attributes associated with the drug. Finally, to manage repeatable prescriptions, we created a third entity, **Repeat**, which contains the relevant attributes (such as the intended interval and maximum repetitions). Alternatively, we could have included the **intended_interval** and **max_times** attributes directly within the **Prescribe** entity, setting them to **None** when they are not needed.

b) From ER-Diagram to SQL_tables: Once the schema was defined, we moved on to the logical schema, followed by defining the tables that will be implemented in SQLAlchemy. As we can observe, due to the 1-N relationship with **Prescription** entity set, keys for **Drug**, **Repeat**, **Internal_Doctor**, and **Visit** have been added. Additionally, the attribute **patient_birthdate** was removed because it is a derived attribute, which can be obtained from the patient's ID.
As for **Drug** and **Repeat**, we simply included the attributes that were present in the ER diagram.

Prescription	
id	Integer
presc_date	Date
patient_id (FK)	Integer
doctor_id (FK)	Integer
quantity	Integer
direction	String
dosage	Integer
drug_id (FK)	Integer
repeat_id (FK)	Integer
visit_id (FK)	Integer

Repeat	
id	Integer
max_times	Integer
intervall	Integer

Drug	
id	Integer
drug_name	String
strenght	Integer

c) Update Code: The next step was to add the new entity sets into our Python file for SQLAlchemy. As first, the corresponding SQL files in the **db** folder were updated to create, insert, and drop also the new tables. Finally, the new models SQLAlchemy ORM in the file **models** were added to allow interaction with the relational database.

d) Perform some test queries: To manage and test all aspects related to the SQL database, a series of queues and functions were created. In total, there are 7 queues, of which 4 are related to the existing part of the database, and the remaining 3 pertain to the new section added during the exercise.

- **find_patient_from_doctor () :** Finds patients who have received a prescription from Dr. Rose Taylor.
- **list_all_prescriptions ():** Lists all prescriptions with details about patients, doctors, and drugs.
- **find_doctors_prescriptions_count ():** Counts the number of prescriptions made by each internal doctor.

For managing all the SQL-related functions, the Python file medical_queries.py is used. In this file, we have the queries and function that execute the three SQL files (insert, drop, create) to interact with the database.

Additionally, the models.py file contains the classes for each entity set. These classes define the structure of the tables in the database and provide the mapping for SQLAlchemy's Object-Relational Mapping (ORM).

Assignment .2 – Data migration

a) Populate Mongo with some sample data: The idea behind this part is to manage the **Prescription** entity set using a different structure other than SQL, specifically with MongoDB (NoSQL). The first step was to define the attributes that will be present in the **Prescription** document. To handle the fact that some prescriptions are repeatable and others are not, we leverage the flexibility offered by MongoDB. The **max_times** and **interval** attributes are only included in the document when necessary. This approach allows us to manage what was previously two separate sets in SQL within a single document. In terms of code, this is implemented through the function **migrate_prescriptions_to_mongodb**, the first step involves reading all the prescriptions from the SQL database. For each prescription, a corresponding document is created. Before inserting the document into MongoDB, a check is made to see if it is already present in the MongoDB database. This helps determine whether the document should be inserted directly or if it needs to be checked for changes (if anything within the prescription has been updated).

```
Already inserted ID: 11
Already inserted ID: 12
Already inserted ID: 13
Updated prescription ID: 14
Inserted prescription ID: 16
```

This implementation allows us to track whether there have been changes or if new prescriptions have been added every time the MongoDB database is updated. This ensures that the migration process is efficient and avoids unnecessary duplicates while keeping the data synchronized between SQL and MongoDB. A peculiar feature within the function is the transformation of the **presc_date** from a **datetime.time** type to a **datetime.datetime** type. This transformation was necessary because MongoDB throws errors if an attempt is made to insert a value of type **datetime.time**.
To avoid inserting incorrect data, the **datetime.time** object is combined with a default time (00:01) to convert it into a **datetime.datetime** object.

b) Perform some test queries: As with the SQL part, after inserting data into the database, queues were created to ensure everything works correctly. In total, 2 queues were created:

- **find_doctor_prescription_count ():** This queue is similar to the one created for SQL, but the key difference lies in how data is stored and processed in MongoDB. Since MongoDB only contains the **IDs** of doctors, patients, and visits (and not the actual names or surnames), an additional queue is included within the MongoDB queue to retrieve and print the names and surnames of the doctors.
- **Find_patient_from_doctor_mongo ():** In this case, the queue is the same as the one implemented for SQL, with the difference being the use of two different databases: SQL and MongoDB. The structure and logic of the queue are similar, but they operate on different databases. Also in this case we have an internal queue to manage the name and surname for patient.

All the functions for managing the MongoDB part are contained within the Python file **pyMongo.py**. Inside this file, we have: queues,function for managing database connections and the function for data migration.

c) Code python: To manage the entire project, four main files are used:

- **Medical_queries.py:** This file contains all the functions for managing SQL-related operations.
- **Models.py:** This file manages the SQLAlchemy ORM for the different entity sets.
- **pyMongo:** This file contains all the functions related to MongoDB.

- **Test_main:** This file is used to manage the execution of the different functions we have in the project. The intention is to control everything through commands entered after invoking the program from the terminal.
  For **example**, running **python3 .\test_main.py info** will display the various options available for the program.

It is important to ensure that the SQL database is populated with data and the migration to MongoDB has been completed before running the queues. This ensures that no errors related to missing data occur.

Assignment .3 – Comparison

To compare the SQL solution with the MongoDB (NoSQL) solution, several aspects will be analyzed, including functionality, complexity and performance.

a) Functionality:
SQL databases are designed for applications that require strong consistency, defined relationships between entities, and complex queries. They store data in tables and use **Primary Keys** and **Foreign Keys** to maintain relationships across different tables. SQL databases are ideal for applications with fixed schemas. Instead MongoDB, as a **document-based NoSQL database**, stores data in **JSON-like documents** that can have nested structures. This is advantageous for applications where the data schema is flexible or evolving over time. In our case, we have entity sets that are fixed with strong relationships, and for this scenario, using an SQL solution would be optimal. However, managing prescriptions is different. In this case, we have a set where the schema changes depending on whether it is repeatable or not. As we can see, the SQL implementation requires the use of an additional entity set, whereas with MongoDB, we can generate a single type of document that changes based on whether the prescription is repeatable or not. This approach allows us to simplify the process and save space.

b) Complexity

SQL databases are **more complex** to set up and maintain due to their rigid schema. Tables need to be carefully defined, and changes to the schema, such as adding new columns or modifying existing ones, often require database migrations. For large applications with complex relationships, designing an efficient schema requires careful thought. NoSQL databases, in particular MongoDB, are **less complex** to set up compared to SQL. They allow for easier development and iterative changes to the data model, as there is no rigid schema. However, this flexibility can lead to inconsistencies in the data if not managed properly, as NoSQL databases do not enforce relationships or constraints. In the case of our project, in terms of code/queue complexity and database management, there are no significant differences between the two solutions. The only notable difference is that when inserting prescriptions into the SQL part, it takes longer because we also need to manage relationships with other entity sets.

c) Performance

SQL databases can handle large volumes of data efficiently but may face performance bottlenecks when handling extremely high-velocity or unstructured data. NoSQL databases are generally **better suited for horizontal scalability** and high-velocity data, making them ideal for applications with unstructured data or where fast read/write operations are required. To analyze the performance, two functions were created to compare the time required by each approach to insert a prescription into the database and to execute the same queue. The results are as follows:

```
Testing MongoDB insert time...
Time to insert into MongoDB: 55.7253360748291 milliseconds
Inserted prescription has been deleted from MongoDB.

Testing SQL_Alchemy insert time...
Time to insert into SQLAlchemy (SQL): 963.7327194213867 milliseconds
Inserted prescription has been deleted from SQL database.
```

```
Testing MongoDB queue time...
Oldest Prescription ID: 14
Doctor ID: 2, Patient ID: 4
Time to execute queue into MongoDB: 23.662090301513672 milliseconds

Testing SQL_Alchemy queue time...
Oldest Prescription ID: 14
Doctor ID: 2, Patient ID: 4
Time to execute queue into SQLAlchemy: 1085.2954387664795 milliseconds
```

As we can see in both tests, MongoDB has significantly lower timings compared to SQLAlchemy. To test the insertion of a prescription, a non-repeatable prescription was inserted to avoid involving other entity sets. This ensured that the test focused only on the data within a single entity set, minimizing the complexity of the operation. Regarding the queue test, the same approach was used—finding the ID of the oldest prescription. This queue does not require data from other entity sets because it operates solely within the prescription entity, a queue like this was implemented to avoid having to retrieve data from SQL in order to perform a queue on MongoDB, since we do not have the entire ER schema in MongoDB. This would compromise the timing test.

Conclusion:

In general, when building a large database, the best approach is to divide it using different technologies, as seen in our case, using SQL for the fixed part and MongoDB for the part that requires flexibility. This reasoning can also be applied to parts that require frequent reads/writes. These parts could benefit from being stored in NoSQL (due to efficiency and performance), while data that will not be accessed or modified frequently might be better stored in SQL.
One important observation made during the project is that when working with a split database, it is crucial to maintain data consistency. Whenever data is added, modified, or deleted in one part, we must ensure it is updated in the other part as well. If this is not done, we may encounter errors in the queues due to missing data, duplicates, or other inconsistencies.
A good practice to address this is to implement a **migration function**, which updates MongoDB based on the data present in SQL. This function ensures that MongoDB stays synchronized with the SQL database. If this kind of update were done every time a queue is executed, we could avoid issues like this, ensuring that data is consistent across both databases and preventing errors related to outdated or inconsistent information.

Regarding the positive and negative aspects of the two approaches:
For SQL, it allows for better organization of data and helps avoid redundancy due to its structured nature. It enforces strict relationships between tables, ensuring data integrity and consistency. However, this structure comes at the cost of flexibility, which makes the management of prescriptions more complex. SQL requires handling relationships across different entity sets, and adding new fields or modifying the schema can be cumbersome. This rigidity may slow down development, especially when the data model needs to evolve.

On the other hand, MongoDB offers greater flexibility in managing prescriptions because of its schema-less design, making it easier to adapt and modify data as needed. This simplicity and efficiency make managing prescriptions more straightforward, especially when dealing with data that might change over time. However, the flexibility that MongoDB offers also introduces potential downsides. If additional documents need to be added, especially with relationships between them, the lack of a well-defined schema in MongoDB can lead to inconsistencies in the data. Without strict enforcement of relationships and constraints, data redundancy and discrepancies could arise, making the system harder to maintain and ensuring that data integrity is not guaranteed.